

CS 3502 Project 1: Multi-Threaded Programming

Lizbeth Ochoa

March 7, 2026

1 Phase 1 : Race Condition

1.1 Program Description

Phase 1 demonstrates race conditions using multiple teller threads in an accounting system. The teller threads perform deposits and withdrawals on a shared bank accounts with synchronization.

1.2 Program Outputs

1.3 Reflection Questions

Q1.1 - Race Condition Evidence

Run	Final total Balance
1	2130
2	1791
3	1924
4	1782
5	2069

Answer: After running the program 5 times, it is clear that there is a different total each time. The program produces a different total because multiple threads simultaneously access and modify the shared account balances. This is without synchronization. The race condition occurs when the threads read and update the same memory at the same time. The critical section occurs in these lines of code:

```
double current_balance = accounts[account_id].balance;  
accounts[account_id].balance = current_balance + amount;
```

Q1.2 Thread vs Process

Answer: A process is its own program with its own memory space and resources. While a thread is a smaller unit of execution within a process. Threads share the same memory space as other threads in the same process which is what connects to race conditions.

Q1.3 pthread_join Purpose

Answer: When I removed the pthread_join calls and ran my program, the main thread terminates and does not wait for the others. So the program becomes an incomplete output. The call is necessary because it makes sure that the main function waits for the rest of the threads to finish.

Q1.4 Non-Determinism

Answer: Non-deterministic behavior means that a program with the same inputs produces different results each time it is ran. This occurs in concurrent programs because the scheduler is what determines which thread executes first, which leads to different results. This makes debugging difficult because an error could not be easily noticed.

2 Phase 2 : Mutex Synchronization

2.1 Program Description

Phase 2 improved our program from Phase 1 by including mutex locks to protect shared data. Each bank account now has mutex locks which a thread must acquire before modifying the account balance. After acquiring the lock and making their changes to the account, the lock is released. This prevents race conditions and ensures only one thread at a time can change an account.

2.2 Reflection Questions

Q2.1 Correctness Verification

Answer: After running the program 10 times, the total balance remained the same each time. This shows how the race conditions from phase 1 was resolved. In Phase 1, we recieved different totals each time we ran the program which showed how each thread raced and caused the data to be inconsistent. In Phase 2, the mutex locks do not allow that and only allow one thread at a time. This ensured more consistent results from our program.

Q2.2 Critical Section

Answer: A critical section is the part of a code where shared resources are accessed or modified. If multiple threads enter this section at the same time, race conditions occur. In Phase 2 program, the critical sections occurred in side the functions of deposit safe and withdrawal safe. These sections include how the account balances were updated. The mutex locks were inserted before and after to make sure a lock is aquired and released before entering the critical section.

Q2.3 Performance Tradeoffs

Answer: Phase 2 program runs slower because of the mutex locks. Each thread must wait for a lock to be aquired after another thread has released the lock. This causes a small wait although is worth it. This is acceptable because it outputs correct results from the program which is needed more than maximum speed. The shared resources must be protected so mutiple threads do not alter it at the same time causing incorrect data.

Q2.4 Granularity

Answer: Each account has its own mutex to allow multiple threads to alter each account simultaneously. If a signle global mutex were used for all accounts, only one thread could access any account at a time. This would really slow down a program because all threads would have to wait even when they are wanting to alter a different account.

3 Phase 3: Deadlock Creation

3.1 Program Description

Phase 3 demonstrates how a deadlock occurs when multiple threads attempt to lock shared resources. In this program a thread needed to lock the source

and destination, but got stuck when it only had one lock and another thread had the other. Both threads were stuck waiting on each other's lock causing a deadlock.

3.2 Reflection Questions

Q3.1 Coffman Conditions

Answer: Deadlock occurs when all four Coffman conditions are met:

1. Mutual Exclusion Each bank account is protected by a mutex lock. This prevents multiple threads from changing a same account simultaneously.

```
pthread_mutex_lock(&accounts[from_id].lock);
```

2. Hold and Wait A thread would lock one account and then will try to lock another while still having the first lock.

```
pthread_mutex_lock(&accounts[from_id].lock);  
pthread_mutex_lock(&accounts[to_id].lock);
```

3. No Preemption If a thread acquires a lock then it cannot be taken away unless it releases it.

```
pthread_mutex_unlock(&accounts[from_id].lock);
```

4. Circular Wait This is when two threads wait on each other in a circular chain. It prevents either thread from continuing.

- Thread 1 holds Account 0 and waits for Account 1
- Thread 2 holds Account 1 and waits for Account 0

Q3.2 Resource Allocation Graph

Answer: Resource allocation graph:

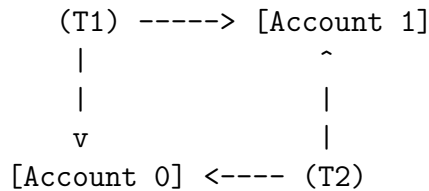
- Threads are represented as circles.
- Accounts are represented as squares.
- Holds and waits are represented as arrows.

Deadlock scenario:

- Thread 1 holds Account 0 and waits for Account 1

- Thread 2 holds Account 1 and waits for Account 0

Graph representation:



Q3.3 Deadlock vs Livelock

Answer: Deadlock occurs when two or more threads are permanently blocked waiting for resources that the others hold. In a deadlock, the threads stop and no progress is made. Livelock is different because the threads are still running but are still reacting. The threads can release and retry the locks but will not be able to complete. Phase 3 only shows a deadlock, it is not considered a livelock because the threads do not release and retr

Q3.4 Real-World Example

Answer: One real-world example we do not notice is a deadlock is a busy intersection. Cars are waiting from all points in a circular wait and causes a deadlock. Source: UNP Education. (2025). *Suprising Real-Life Examples of Deadlock in DBMS Explained*.

4 Phase 4 : Deadlock Resolution

4.1 Program Description

In phase 4, deadlock is prevented using the lock ordering strategy. “

4.2 Reflection Questions

Q4.1 Breaking Circular Wait

Answer: Lock ordering prevents deadlock by making all threads acquire the locks in order. In our program, the lower lock is aquired before the higher

one. This eliminates the Circular wait Coffman condition because all threads have to follow the same order.

Q4.2 Strategy Comparison

Answer: To start with lock ordering, it is simple and does not require retries or timeouts. It guarantees an order for the threads to reach the locks. As for timeout mechanisms, threads will retry the locks within a certain time limit. This may cause a decrease in performance because of threads constantly trying to acquire locks. Lastly, for try-lock backoff it retries acquiring the lock after waiting for a short time. This only reduces blocking a bit and increased repeated retries.

Q4.3 Starvation

Answer: Lock ordering does not usually cause starvation because the threads eventually will get the needed lock. Although, if there are many threads that are wanting to acquire the lock they could be delayed if another thread acquires it first.

Q4.4 Production Recommendation

Answer: For a real banking system, I would stick with lock ordering. It prevents deadlocks and is the simplest form to do so. The other strategies require retries or timeouts that could potentially cause problems. Since we know that in lock ordering have a simple pattern, it is easier for the program to be maintained.